

Programming for Robotics

Robotics

Microcontrollers vs. Single-Board Computers

Robotics projects are typically built around one of two kinds of onboard computer, and picking the right one depends on what the robot needs to do:

- Microcontrollers (like the Arduino family) — run one program directly on the hardware with no operating system, very reliable and real-time, ideal for directly reading sensors and driving motors with precise timing, but limited processing power.
- Single-board computers / SBCs (like the Raspberry Pi) — run a full operating system (usually Linux), giving access to cameras, networking, and general-purpose programming languages, ideal for higher-level decision-making, vision processing, or Wi-Fi/remote control.
- Many real robots use both together: a microcontroller handles the fast, low-level sensor/motor loop, while an SBC handles the slower, higher-level 'thinking' and hands down commands to the microcontroller.

Programming Microcontrollers (Arduino)

Arduino programs (called 'sketches') are written in a simplified C/C++ and always follow the same two-function structure: a `setup()` function that runs once at power-on to configure pins and initial state, and a `loop()` function that repeats continuously for as long as the board has power. This structure directly mirrors the sense-think-act cycle: each pass through `loop()` reads sensors, makes a decision, and updates actuators before repeating.

- `pinMode()` configures a pin as INPUT (reading a sensor) or OUTPUT (driving an LED, motor driver, etc.).
- `digitalRead()/digitalWrite()` handle simple on/off signals; `analogRead()/analogWrite()` handle variable voltages (like a potentiometer or motor speed via PWM).
- `delay()` pauses execution for a set time — useful for simple sequencing, but blocks the whole loop, which becomes a problem once a robot needs to do multiple things at once.
- `millis()` returns the time since the board started, and is the standard way to do timing without blocking — checking 'has enough time passed?' instead of pausing execution.

Programming Single-Board Computers (Raspberry Pi)

A Raspberry Pi runs a general-purpose Linux operating system, so robotics code is usually written in Python (or C++) as a normal program rather than a fixed setup/loop sketch. This gives access to full libraries for computer vision, networking, and file storage that a microcontroller can't realistically support, at the cost of less predictable timing — the operating system can briefly delay a program at any point, which matters for tasks needing exact real-

time control (better left to a microcontroller working alongside it).

Basic Control Loops

A control loop is a repeating cycle that continuously compares a robot's current state to a desired state, and adjusts its actuators to close the gap. The simplest useful version is the on-off (bang-bang) controller: if a sensor reading is above a threshold, do one thing; if below, do another — this is exactly the obstacle-avoidance example from the sensors module.

- Open-loop control: the robot executes a fixed sequence of actions without checking sensor feedback at all (e.g. 'drive forward for 3 seconds'). Simple, but drifts and errors accumulate with nothing correcting them.
- Closed-loop control: the robot continuously measures its actual state via sensors and adjusts its actions based on the difference from the target — far more robust to real-world variation like uneven floors or motor differences.
- Proportional (P) control: a step up from bang-bang control — instead of a full on/off response, the correction applied is proportional to how far off the current state is from the target, giving smoother, less jerky behavior (e.g. slowing down gradually as a line-following robot approaches the line's edge, rather than snapping hard left or right).

A Complete Example: Line-Following Robot

Bringing together sensors, actuators, and a control loop: a line-following robot uses two IR reflectance sensors mounted at the front, pointed down. In each pass of the control loop, it reads both sensors; if the left sensor detects the line and the right doesn't, it steers right motor faster to correct back toward the line, and vice versa; if both sensors detect the line, it drives both motors evenly forward. This is the same three-part sense-think-act structure introduced in the first module, now expressed as working, repeatable code — the pattern every more advanced robotics project in this course builds on.

Learn More

Arduino Official Documentation

<https://docs.arduino.cc/>

Raspberry Pi Official Documentation

<https://www.raspberrypi.com/documentation/>

freeCodeCamp YouTube Channel

<https://www.youtube.com/@freecodecamp>